POKHARA UNIVERSITY

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

LABORATORY REPORT

AGILE SOFTWARE DEVELOPMENT LAB

Lab No. 5

BESE III YEAR - VI SEM

Submitted By:

Pranish Poudel
Roll No: 26
Program: BESE ,2023

Submitted To:

Er. Rajendra Bahadur Thapa
Lecturer

Date: July 3, 2026

NAME OF EXPERIMENT

Building a CI/CD pipeline with GitHub Actions and exploring Jenkins.

AIM

To set up a Continuous Integration and Continuous Deployment (CI/CD) pipeline using GitHub Actions, and to study the features and operation of Jenkins as a CI/CD automation server.

THEORY

Continuous Integration (CI) and Continuous Deployment (CD) are core DevOps practices that automate the software delivery lifecycle. They let teams integrate code often, run automated tests, and deploy reliably and quickly. A CI/CD pipeline cuts manual work, reduces human error, and tightens the feedback loop between development and operations.

1. Continuous Integration (CI): CI is the practice of automatically building and testing code every time a change is pushed to a shared repository. Developers commit frequently, often several times a day, and each commit triggers an automated build and test run that confirms the new code integrates cleanly with the existing codebase.

- Integration bugs and conflicts are caught early.
- Automated tests ensure new changes do not break existing behaviour.
- Immediate feedback shortens development cycles.
- Consistent testing keeps code quality high.
- Manual build and test effort is reduced.

2. Continuous Deployment (CD): CD extends CI by automatically deploying the application to staging or production once it passes all automated tests. Removing the manual deployment step lets teams ship features, fixes, and updates rapidly and dependably.

- Releases become faster and more frequent.
- Automation reduces deployment errors.
- New features and improvements are delivered continuously.
- Reliability improves through a consistent deployment process.
- Teams respond to customer feedback more quickly.

3. GitHub Actions: GitHub Actions is a CI/CD platform built directly into GitHub. Workflows are written in YAML and can be triggered by events such as pushes, pull requests, releases, or schedules, all without external tooling.

- Workflow automation: custom workflows for building, testing, and deploying.
- Actions Marketplace: reusable actions for common tasks.
- Matrix builds: test across several operating systems, languages, and versions at once.
- Event-driven: workflows respond to GitHub events.
- Cross-platform: runners for Windows, Linux, and macOS.
- Secrets management: secure storage of keys and tokens.

4. Jenkins: Jenkins is an open-source automation server that has anchored CI/CD for over a decade. It is highly extensible through a large plugin ecosystem covering Git, Docker, Kubernetes, cloud platforms,

and testing tools, and it supports both declarative and scripted pipelines.

- Open-source and free, with a large active community.
- Pipeline as code using a Groovy-based DSL.
- A vast plugin ecosystem for integrating with almost any tool.
- Distributed builds across multiple agent nodes.
- Integration with testing frameworks for automated tests.
- A web dashboard for monitoring builds and managing jobs.

Together, CI/CD practices and tools such as GitHub Actions and Jenkins let teams adopt DevOps principles, improve collaboration between development and operations, and deliver quality software at speed.

OBJECTIVES

- To understand the fundamentals of Continuous Integration and Continuous Deployment.
- To build a working CI/CD pipeline with GitHub Actions for automated build and testing.
- To automate builds and test runs using YAML-defined workflows.
- To trigger workflows from repository events such as a push.
- To study the architecture, features, and capabilities of Jenkins.
- To appreciate the role of automation in Agile and DevOps development.

ALGORITHM

1. Create a GitHub repository for the sample project.
2. Develop a simple application (for example in Node.js) and push it to the repository.
3. Ensure the application has a runnable test suite such as Jest.
4. Create a .github/workflows directory inside the repository.
5. Add a YAML workflow file (for example ci.yml) in that directory.
6. Configure the workflow triggers, such as a push to the main branch.
7. Define a job with a runner environment such as ubuntu-latest.
8. Add steps: check out the code, set up the language, install dependencies, and run tests.
9. Commit and push the workflow file to GitHub.
10. Open the Actions tab to watch the workflow run.
11. Confirm the workflow succeeds on each push and that all tests pass.
12. Study Jenkins conceptually through its architecture, plugins, and pipelines.
13. Document the observations and compare GitHub Actions with Jenkins.

CODE

GitHub Actions workflow (.github/workflows/ci.yml)

```
name: Node.js CI

on:
  push:
    branches:
```

```
      - main
  pull_request:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [18, 20]

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}

      - name: Install Dependencies
        run: npm install

      - name: Run Tests
        run: npm test

      - name: Display Success Message
        run: echo "Build and tests completed successfully!"
```

Sample application (app.js)

```
function add(a, b) {
    return a + b;
}

function subtract(a, b) {
    return a - b;
}

function multiply(a, b) {
    return a * b;
}

module.exports = { add, subtract, multiply };
```

Sample test file (app.test.js)

```
const { add, subtract, multiply } = require('./app');

test('Addition', () => {
    expect(add(2, 3)).toBe(5);
});

```

```
test('Subtraction', () => {
    expect(subtract(10, 4)).toBe(6);
});

test('Multiplication', () => {
    expect(multiply(3, 7)).toBe(21);
});
```

package.json (scripts section)

```
"scripts": {
    "test": "jest"
}
```

OUTPUT

The GitHub Actions workflow ran successfully. The following was recorded.

- The runner cloned the repository automatically.
- Node.js versions 18 and 20 were configured across the matrix build.
- Dependencies installed without errors.
- All three test cases passed.
- The workflow completed after every push to the main branch.
- The Actions dashboard showed a green check mark for the successful run.
- The run took roughly 45 seconds.
- Jenkins was understood as a robust, plugin-extensible server for enterprise-grade pipelines.

RESULT

The CI/CD pipeline was implemented successfully with GitHub Actions. On every push the workflow built the project, installed dependencies, and ran the test suite, and all tests passed, confirming the application is correct. Jenkins was additionally explored as a flexible automation server that supports advanced pipelines, distributed builds, and extensive plugin-based integrations.

CONCLUSION

This experiment gave hands-on experience of CI/CD automation with GitHub Actions. The automated workflow greatly reduced the manual effort of building and testing while catching integration problems early. The study of Jenkins highlighted its role as an industry-standard tool for complex, enterprise-grade pipelines through its plugin ecosystem and pipeline-as-code approach. Understanding CI/CD is essential in modern software development, letting teams deliver high-quality software quickly, reliably, and with little manual overhead.